



UNIVERSIDAD DE ANTIOQUIA

1 8 0 3

Laboratorio de Sistemas Operativos Gestión de Memoria Practica No. 3

**Ricardo Medina Herrera
Santiago Villegas Naranjo**

Espacio de direcciones

1.1 Actividad: Programa base

1.2 Actividad: Visualizar los mapas de memoria de un proceso

1.3 Actividad: Exploración de /proc/[pid]/maps

1.4 Actividad: Comparar espacios de dos procesos simultáneos

[API de Memoria](#)

[2.1 Actividad: Programa base](#)

[2.2 Actividad: Uso correcto de malloc y free](#)

[2.3 Actividad: Código con bugs de memoria](#)

[2.4 Actividad: Identificar y corregir errores de memoria](#)

[Traducción de direcciones — Base & Bounds](#)

[3.1 Actividad: Simulador](#)

[3.2 Actividad: Base & Bounds — Análisis](#)

[Segmentación](#)

[4.1 Traducción manual con tabla de segmentos](#)

[Paginación](#)

[5.1 Actividad: Cálculo de la tabla de páginas](#)

[5.2 Actividad: Simulador de paginación](#)

[5.3 Actividad: Simulador — Análisis](#)

[Gestión de espacio libre](#)

[6.1 Actividad: Simulación de estrategias de asignación](#)

[6.2 Actividad: Fragmentación](#)

[6.3 Actividad: Fragmentación en glibc — Análisis](#)

[TLBs — Translation Lookaside Buffer](#)

[7.1 Actividad: Localidad y TLB — Análisis](#)

[7.2 Actividad: Comportamiento de los TLB](#)

Espacio de direcciones

1.1 Actividad: Programa base

Se nos da el siguiente archivo

```
C/C++
// mem_map.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 42; /* segmento de datos (.data) */

int main() {
    int local_var = 10; /* stack */
    int *heap_var = malloc(sizeof(int) * 100); /* heap */
}
```

```

*heap_var = 99;

printf("PID del proceso : %d\n", getpid());
printf("Dir. codigo (main) : %p\n", (void *)main);
printf("Dir. global_var : %p\n", (void *)&global_var);
printf("Dir. local_var : %p\n", (void *)&local_var);
printf("Dir. heap_var : %p\n", (void *)heap_var);

printf("\nPresione ENTER para continuar...\n");
getchar();
free(heap_var);
return 0;
}

```

Se compilo el archivo a un ejecutable con el mismo nombre del archivo pero sin la extensión

1.2 Actividad: Visualizar los mapas de memoria de un proceso

Se ejecutó el ejecutable del programa mem_map y luego se ejecutaron los comandos detallados en la guía obteniendo el siguiente resultado

Ejecución del programa

Salida

```

None
./mem_map
PID del proceso : 13271
Dir. codigo (main) : 0x5642404080b0
Dir. global_var : 0x56424040b010
Dir. local_var : 0x7ffe3104e6a4
Dir. heap_var : 0x56427c0922a0

Presione ENTER para continuar...

```

Comando ejecutado:

```

None
cat /proc/$(pgrep mem_map)/maps

```

Salida del comando

None

```
cat /proc/$(pgrep mem_map)/maps
564240407000-564240408000 r--p 00000000 00:1a 6262837
/home/docair/Documents/octavo-semester/so/labs/tercero/1-espacio-direcciones/mem_m
ap
564240408000-564240409000 r-xp 00001000 00:1a 6262837
/home/docair/Documents/octavo-semester/so/labs/tercero/1-espacio-direcciones/mem_m
ap
564240409000-56424040a000 r--p 00002000 00:1a 6262837
/home/docair/Documents/octavo-semester/so/labs/tercero/1-espacio-direcciones/mem_m
ap
56424040a000-56424040b000 r--p 00002000 00:1a 6262837
/home/docair/Documents/octavo-semester/so/labs/tercero/1-espacio-direcciones/mem_m
ap
56424040b000-56424040c000 rw-p 00003000 00:1a 6262837
/home/docair/Documents/octavo-semester/so/labs/tercero/1-espacio-direcciones/mem_m
ap
56427c092000-56427c0b3000 rw-p 00000000 00:00 0 [heap]
7fae20c00000-7fae20c28000 r--p 00000000 00:1a 1580057
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/libc.so.6
7fae20c28000-7fae20da6000 r-xp 00028000 00:1a 1580057
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/libc.so.6
7fae20da6000-7fae20df5000 r--p 001a6000 00:1a 1580057
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/libc.so.6
7fae20df5000-7fae20df9000 r--p 001f4000 00:1a 1580057
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/libc.so.6
7fae20df9000-7fae20dfb000 rw-p 001f8000 00:1a 1580057
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/libc.so.6
7fae20dfb000-7fae20e08000 rw-p 00000000 00:00 0
7fae20ecf000-7fae20ed4000 rw-p 00000000 00:00 0
7fae20ed4000-7fae20ed8000 r--p 00000000 00:00 0 [vvar]
7fae20ed8000-7fae20eda000 r-xp 00000000 00:00 0 [vdso]
7fae20eda000-7fae20edb000 r--p 00000000 00:1a 1580051
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/ld-linux-x86-64.so.
2
7fae20edb000-7fae20f05000 r-xp 00001000 00:1a 1580051
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/ld-linux-x86-64.so.
2
7fae20f05000-7fae20f0f000 r--p 0002b000 00:1a 1580051
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/ld-linux-x86-64.so.
2
```

```
7fae20f0f000-7fae20f11000 r--p 00035000 00:1a 1580051
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/ld-linux-x86-64.so.
2
7fae20f11000-7fae20f13000 rw-p 00037000 00:1a 1580051
/nix/store/vr7ds8vwb12fz7pr221d5y0f8n9a5wda-glibc-2.40-218/lib/ld-linux-x86-64.so.
2
7ffe31030000-7ffe31054000 rw-p 00000000 00:00 0 [stack]
```

Comando ejecutado

```
None
pmap -x $(pgrep mem_map)
```

Salida del comando

```
None
13271:  ./mem_map
Address      Kbytes      RSS      Dirty Mode  Mapping
0000564240407000      4         4         0 r---- mem_map
0000564240408000      4         4         0 r-x-- mem_map
0000564240409000      4         4         0 r---- mem_map
000056424040a000      4         4         4 r---- mem_map
000056424040b000      4         4         4 rw--- mem_map
000056427c092000    132         4         4 rw--- [ anon ]
00007fae20c00000    160        156         0 r---- libc.so.6
00007fae20c28000   1528       1056         0 r-x-- libc.so.6
00007fae20da6000    316        128         0 r---- libc.so.6
00007fae20df5000     16         16        16 r---- libc.so.6
00007fae20df9000      8          8         8 rw--- libc.so.6
00007fae20dfb000     52         20        20 rw--- [ anon ]
00007fae20ecf000     20         16        16 rw--- [ anon ]
00007fae20ed4000     16          0         0 r---- [ anon ]
00007fae20ed8000      8          4         0 r-x-- [ anon ]
00007fae20eda000      4          4         0 r---- ld-linux-x86-64.so.2
00007fae20edb000    168       168         0 r-x-- ld-linux-x86-64.so.2
00007fae20f05000     40         40         0 r---- ld-linux-x86-64.so.2
00007fae20f0f000      8          8         8 r---- ld-linux-x86-64.so.2
00007fae20f11000      8          8         8 rw--- ld-linux-x86-64.so.2
00007ffe31030000    144         24        24 rw--- [ stack ]
-----
total kB          2648      1680      112<
```

1.3 Actividad: Exploración de /proc/[pid]/maps

1. Identifique en la salida de /proc/maps las regiones text, heap y stack. ¿Qué permisos (r/w/x/p) tiene cada una? ¿Por qué difieren?

Región text: Tiene permisos de lectura y ejecución

None

```
564240408000-564240409000 r-xp 00001000 00:1a 6262837  
/home/docair/Documents/octavo-semester/so/labs/tercero/1-espacio-direcciones/mem_m  
ap
```

La región text contiene el código ejecutable del programa, incluyendo la función main. Tiene permisos r-xp porque el código debe poder leerse y ejecutarse, pero no modificarse durante la ejecución del proceso.

Región heap: Tiene permisos de lectura y escritura

None

```
56427c092000-56427c0b3000 rw-p 00000000 00:00 0 [heap]
```

La región heap almacena memoria dinámica reservada mediante malloc(), por lo que necesita permisos de lectura y escritura. No posee permisos de ejecución porque allí no se ejecuta código.

Región stack: Tiene permisos de lectura y escritura

None

```
7ffe31030000-7ffe31054000 rw-p 00000000 00:00 0 [stack]
```

La región stack almacena variables locales, parámetros y llamadas a funciones. Tiene permisos de lectura y escritura porque su contenido cambia constantemente durante la ejecución del programa. Tampoco posee permisos de ejecución debido a que no está destinada para ejecutar instrucciones.

Nota: La región de text no tiene permisos de escritura porque es un binario y no tiene sentido que se pueda modificar el binario, las regiones de heap y stack no tienen el permiso de ejecución porque

2. Compare las direcciones impresas con los rangos de /proc/maps. ¿A qué región pertenece cada variable?

Las direcciones impresas por el programa corresponden a distintas regiones del espacio de memoria virtual del proceso.

La función main tiene dirección:

0x5642404080b0

y pertenece a la región text del ejecutable, ya que coincide con el rango ejecutable del programa mostrado en /proc/maps.

La variable global_var tiene dirección:

0x56424040b010

y pertenece al segmento de datos (.data), correspondiente a una región con permisos de lectura y escritura del ejecutable.

La variable local_var tiene dirección:

0x7ffe3104e6a4

y pertenece a la región stack, debido a que es una variable local declarada dentro de la función main.

La variable heap_var tiene dirección:

0x56427c0922a0

y pertenece a la región heap, ya que fue reservada dinámicamente mediante malloc().

3. ¿Qué otras regiones aparecen en el mapa ('libc', '[vdso]', '[vsyscall]')? ¿Qué función cumple cada una?

- libc: En esta región se encuentra la biblioteca estándar de C, la librería de C GNU, aquí podemos encontrar funciones esenciales como printf, malloc, free entre otros
- ld-linux-x86-64.so.2: Es el dynamic linker / loader de Linux, será el primero en ser llamado por el sistema operativo, su trabajo será obtener las bibliotecas y resolver los símbolos dinámicos antes de ejecutar el main, por ejemplo al ver que nuestro programa usa funciones que dependen de libc se encargará de cargarlo
- Virtual Dynamic Shared Object(vdso): Los llamados al sistema son costosos por operaciones que implican como cambios parciales de contexto, entonces para hacer llamados de sistema para funciones más simples se utiliza esta región, una operación de ejemplo sería obtener el tiempo de esta forma se tiene un mejor rendimiento

- Virtual variables(vvar): Esta región complementa a vdso porque en esta el sistema operativo compartirá información el proceso como el tiempo actual, para que luego cuando el proceso solicite esa información con un llamado al sistema en lugar de hacer un llamado al sistema la región vdso usa esta para obtener la información
- anon: Es una región similar al heap pero aquí se almacenan las regiones creadas utilizando mmap a diferencia que en el heap donde se guarda lo asociado al malloc
- vsyscall: no apareció en la ejecución de los comandos

4. ¿Son las direcciones virtuales iguales a las físicas? Explique apoyándose en el concepto de address space del OSTEP.

No, las direcciones observadas corresponden a direcciones virtuales y no a direcciones físicas reales de la memoria RAM. Según el concepto de address space explicado en OSTEP, cada proceso posee su propio espacio de direcciones virtual privado y aislado. El sistema operativo crea la ilusión de que cada proceso tiene memoria continua y única.

Internamente, la memory management unit y las tablas de páginas traducen las direcciones virtuales utilizadas por el proceso hacia direcciones físicas reales en memoria. Gracias a este mecanismo, diferentes procesos pueden utilizar espacios virtuales sin interferir entre sí.

1.4 Actividad: Comparar espacios de dos procesos simultáneos

Primera instancia del programa

```
None
PID del proceso : 7082
Dir. codigo (main) : 0x557a45608199
Dir. global_var : 0x557a4560b048
Dir. local_var : 0x7ffe6803aaac
Dir. heap_var : 0x557a4acf92a0

Presione ENTER para continuar...
```

Segunda instancia del programa

```
None
PID del proceso : 8332
Dir. codigo (main) : 0x5644e72c3199
Dir. global_var : 0x5644e72c6048
Dir. local_var : 0x7ffc08a48aec
Dir. heap_var : 0x56450228a2a0
```


Presione ENTER para continuar...

1. ¿Son las mismas direcciones virtuales en ambos procesos? ¿Qué conclusión saca sobre el aislamiento del espacio de direcciones?

No. Aunque ambos ejecutan el mismo programa, cada proceso posee su propio espacio de direcciones virtuales independiente. El sistema operativo, mediante la MMU y el proceso de traducción, traduce las direcciones virtuales de cada proceso a distintas direcciones físicas de memoria. Esto garantiza el aislamiento entre procesos, evitando que un proceso pueda acceder directamente a la memoria de otro.

2. ¿Podría el Proceso A leer o modificar la variable global del Proceso B mediante su dirección virtual? Justifique.

No, el proceso A no puede acceder directamente a la variable global del proceso B usando su dirección virtual. Ya que las direcciones virtuales solamente tienen validez dentro del espacio de direcciones del proceso propietario, aunque ambos procesos tengan variables similares, el sistema operativo mantiene aislamiento y protección entre procesos mediante memoria virtual y tablas de páginas.

Por esta razón, una dirección virtual perteneciente al proceso B no puede ser utilizada directamente por el proceso A para leer o modificar memoria de otro proceso.

API de Memoria

2.1 Actividad: Programa base

Comando ejecutado

```
None  
gcc -Wall -o heap_demo heap_demo.c
```

El comando no produjo una salida

Comando ejecutado

None

```
valgrind --leak-check=full --track-origins=yes ./heap_demo
```

Salida del comando

None

```
==17108== Memcheck, a memory error detector
==17108== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==17108== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==17108== Command: ./heap_demo
==17108==
Arreglo original: 0 1 4 9 16 25 36 49 64 81
Arreglo ampliado: 0 1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256 289 324
361
==17108==
==17108== HEAP SUMMARY:
==17108==      in use at exit: 0 bytes in 0 blocks
==17108==    total heap usage: 3 allocs, 3 frees, 1,144 bytes allocated
==17108==
==17108== All heap blocks were freed -- no leaks are possible
==17108==
==17108== For lists of detected and suppressed errors, rerun with: -s
==17108== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

2.2 Actividad: Uso correcto de malloc y free

1. Muestre la salida completa de Valgrind. ¿Reporta errores o fugas de memoria? ¿Qué significa el mensaje "All heap blocks were freed"?

La ejecución de Valgrind sobre heap_demo no reportó errores ni fugas de memoria.

El resumen mostrado fue:

None

```
HEAP SUMMARY:
    in use at exit: 0 bytes in 0 blocks
    total heap usage: 3 allocs, 3 frees, 1,144 bytes allocated
```

```
All heap blocks were freed -- no leaks are possible
```

```
ERROR SUMMARY: 0 errors from 0 contexts
```

Esto significa que toda la memoria reservada dinámicamente mediante malloc() y realloc() fue correctamente liberada utilizando free(), por lo que no quedaron bloques de memoria ocupados al finalizar el programa.

El mensaje “All heap blocks were freed - - no leaks are possible” indica que no existen memory leaks, es decir, no quedó memoria sin liberar al terminar la ejecución.

2. ¿Por qué se usa sizeof(int) en lugar del valor literal 4? ¿Qué ventaja ofrece en portabilidad entre arquitecturas?
Se hace de esta forma porque aunque para algunas arquitecturas de procesador el valor del tamaño para un entero sea 4 para otras el valor será distinto, al usar sizeof en lugar de dejar un valor fijo nuestro código puede ser ejecutado en distintas arquitecturas sin presentar problemas, dándole portabilidad entre arquitecturas de procesadores.
3. ¿Qué devuelve malloc cuando no hay memoria disponible? ¿Por qué es crítico verificar ese valor antes de usarlo?
Cuando malloc falla este retorna el valor de NULL, es importante verificar que el valor que retorna sea distinto a NULL porque de lo contrario podría suceder que se intente acceder a una dirección inválida causando un error de fragmentación lo que haría colapsar al programa.

2.3 Actividad: Código con bugs de memoria

Comando ejecutado

```
None  
gcc -Wall -g -o buggy_mem buggy_mem.c
```

El comando no produjo una salida

Comando ejecutado

None

```
valgrind --leak-check=full --track-origins=yes ./buggy_mem
```

Salida del comando

None

```
==9799== Memcheck, a memory error detector
==9799== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward
et al.
==9799== Using Valgrind-3.25.1 and LibVEX; rerun with -h for
copyright info
==9799== Command: ./buggy_mem
==9799==
==9799== Invalid write of size 4
==9799==    at 0x400119F: main (buggy_mem.c:10)
==9799==   Address 0x4a5d054 is 0 bytes after a block of size 20
alloc'd
==9799==    at 0x48487C4: malloc (vg_replace_malloc.c:446)
==9799==   by 0x400117A: main (buggy_mem.c:8)
==9799==
hola mundo
==9799== Invalid read of size 4
==9799==    at 0x40011ED: main (buggy_mem.c:19)
==9799==   Address 0x4a5d040 is 0 bytes inside a block of size 20
free'd
==9799==    at 0x484B78B: free (vg_replace_malloc.c:989)
==9799==   by 0x40011E8: main (buggy_mem.c:18)
==9799== Block was alloc'd at
==9799==    at 0x48487C4: malloc (vg_replace_malloc.c:446)
==9799==   by 0x400117A: main (buggy_mem.c:8)
==9799==
p[0] = 0
==9799==
==9799== HEAP SUMMARY:
==9799==    in use at exit: 100 bytes in 1 blocks
```

```

==9799==    total heap usage: 3 allocs, 2 frees, 1,144 bytes
allocated
==9799==
==9799== 100 bytes in 1 blocks are definitely lost in loss record 1
of 1
==9799==    at 0x48487C4: malloc (vg_replace_malloc.c:446)
==9799==    by 0x40011B4: main (buggy_mem.c:13)
==9799==
==9799== LEAK SUMMARY:
==9799==    definitely lost: 100 bytes in 1 blocks
==9799==    indirectly lost: 0 bytes in 0 blocks
==9799==    possibly lost: 0 bytes in 0 blocks
==9799==    still reachable: 0 bytes in 0 blocks
==9799==    suppressed: 0 bytes in 0 blocks
==9799==
==9799== For lists of detected and suppressed errors, rerun with:
-s
==9799== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0
from 0)

```

2.4 Actividad: Identificar y corregir errores de memoria

1. Transcribe los mensajes que arroja Valgrind. ¿Cual mensaje corresponde a cada uno de los tres errores clásicos?

Valgrind detectó tres errores diferentes durante la ejecución del programa buggy_mem.

El primer error corresponde a un buffer overflow:

```

None
Invalid write of size 4
at 0x400119F: main (buggy_mem.c:10)
Address 0x4a5d054 is 0 bytes after a block of size 20 alloc'd

```

Este error ocurre porque el ciclo utiliza la condición $i \leq 5$ en lugar de $i < 5$, escribiendo fuera de

los límites del arreglo reservado dinámicamente.

El segundo error corresponde a un user-after-free :

```
None
Invalid read of size 4
at 0x40011ED: main (buggy_mem.c:19)
Address 0x4a5d040 is 0 bytes inside a block of size 20 free'd
```

Este error sucede porque el programa intenta acceder a `p[0]` después de haber liberado la memoria con `free(p)`.

El tercer error corresponde a un memory leak:

```
None
100 bytes in 1 blocks are definitely lost in loss record 1 of 1
```

Esto ocurre porque la memoria reservada para el puntero que nunca fue liberada mediante `free(q)`.

El resumen final de Valgrind fue:

```
None
ERROR SUMMARY: 3 errors from 3 contexts
```

Confirmando la presencia de los tres errores.

2. Corrija el programa (`buggy_mem_fixed.c`) y verifique con Valgrind que no queda ningún error ni fuga.

Código corregido
Archivo `fixed_buggy_mem.c`

```
C/C++
// buggy_mem.c -- NO ejecutar sin Valgrind
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```

int main() {
    /* ERROR 1: buffer overflow */
    int *p = malloc(5 * sizeof(int));
    for (int i = 0; i < 5; i++) /* <= en vez de < */
        p[i] = i;

    /* ERROR 2: memory leak (nunca se llama free(q)) */
    char *q = malloc(100);
    strcpy(q, "hola mundo");
    printf("%s\n", q);
    free(q);

    /* ERROR 3: use-after-free */
    printf("p[0] = %d\n", p[0]); /* acceso ilegal */
    free(p);

    return 0;
}

```

Comprobación con valgrind

None

```
valgrind --leak-check=full --track-origins=yes ./fixed_buggy_mem
```

Salida obtenido

None

```

==17167== Memcheck, a memory error detector
==17167== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==17167== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==17167== Command: ./fixed_buggy_mem
==17167==
hola mundo
p[0] = 0
==17167==
==17167== HEAP SUMMARY:
==17167==      in use at exit: 0 bytes in 0 blocks
==17167==    total heap usage: 2 allocs, 2 frees, 1,124 bytes allocated
==17167==
==17167== All heap blocks were freed -- no leaks are possible
==17167==
==17167== For lists of detected and suppressed errors, rerun with: -s

```

```
==17167== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

3. ¿Que consecuencias puede tener un use-after-free en un programa real en términos de seguridad y estabilidad del sistema?

Un user-after-free puede provocar fallos graves de estabilidad y seguridad. En términos de estabilidad, puede causar comportamientos impredecibles, corrupción de memoria, resultados incorrectos o cierres inesperados del programa. En términos de seguridad, este tipo de vulnerabilidad puede ser explotada por atacantes para ejecutar código arbitrario, modificar datos sensibles o tomar control parcial del sistema. Debido a esto, los errores use-after-free son considerados vulnerabilidades críticas en software.

Traducción de direcciones — Base & Bounds

3.1 Actividad: Simulador

Aqui solo era agregar el archivo compartido en la guía

3.2 Actividad: Base & Bounds — Análisis

1. Compile y ejecute. Muestre la salida completa. ¿Qué ocurre al acceder a VA=64 y VA=100 en el Proceso A? ¿Que haría el SO real ante esta excepción?

Salida obtenida al ejecutar el programa

```
None
./base_bounds
--- Proceso A (base=32, bounds=64) ---
VA= 0 -> PA= 32
VA= 10 -> PA= 42
VA= 63 -> PA= 95
[EXCEPCION] VA=64 viola bounds=64
[EXCEPCION] VA=100 viola bounds=64
--- Proceso B (base=128, bounds=80) ---
VA= 0 -> PA=128
VA= 10 -> PA=138
VA= 63 -> PA=191
VA= 64 -> PA=192
[EXCEPCION] VA=100 viola bounds=80
```


Al acceder a la VA 64 se comprueba que no sea mayor que el bounds y como no lo es se realiza la traducción de esta a una dirección física obteniendo la PA 192. En el caso de la VA 100 esta es mayor que el bounds por lo que se incumple la condición y se arroja un fallo de protección.

2. Agregue un Proceso C (base=0, bounds=32) al programa y traduzca las mismas VAs. ¿Puede el Proceso A acceder a las direcciones del Proceso C directamente? Justifique.

Programa modificado

```
C/C++
// base_bounds.c
#include <stdio.h>

typedef struct {
    int base;
    int bounds;
} Registro;

/* Traduce VA -> PA; imprime excepcion si viola bounds */
int traducir(Registro r, int va) {
    if (va < 0 || va >= r.bounds) {
        printf(" [EXCEPCION] VA=%d viola bounds=%d\n", va, r.bounds);
        return -1;
    }
    return r.base + va;
}

int main() {
    Registro procA = {32, 64}; /* base=32, bounds=64 */
    Registro procB = {128, 80}; /* base=128, bounds=80 */
    Registro procC = {0, 32}; /* base=0, bounds=32 */
    int vas[] = {0, 10, 63, 64, 100};
    int n = sizeof(vas) / sizeof(vas[0]);

    printf("--- Proceso A (base=%d, bounds=%d) ---\n", procA.base, procA.bounds);
    for (int i = 0; i < n; i++) {
        int pa = traducir(procA, vas[i]);
        if (pa != -1)
            printf(" VA=%3d -> PA=%3d\n", vas[i], pa);
    }
    printf("--- Proceso B (base=%d, bounds=%d) ---\n", procB.base, procB.bounds);
    for (int i = 0; i < n; i++) {
        int pa = traducir(procB, vas[i]);
```

```

    if (pa != -1)
        printf(" VA=%3d -> PA=%3d\n", vas[i], pa);
}
printf("--- Proceso C (base=%d, bounds=%d) ---\n", procC.base, procC.bounds);
for (int i = 0; i < n; i++) {
    int pa = traducir(procC, vas[i]);
    if (pa != -1)
        printf(" VA=%3d -> PA=%3d\n", vas[i], pa);
}
return 0;
}

```

Salida tras ejecución del programa

None

```

./base_bounds
--- Proceso A (base=32, bounds=64) ---
VA=  0 -> PA= 32
VA= 10 -> PA= 42
VA= 63 -> PA= 95
[EXCEPCION] VA=64 viola bounds=64
[EXCEPCION] VA=100 viola bounds=64
--- Proceso B (base=128, bounds=80) ---
VA=  0 -> PA=128
VA= 10 -> PA=138
VA= 63 -> PA=191
VA= 64 -> PA=192
[EXCEPCION] VA=100 viola bounds=80
--- Proceso C (base=0, bounds=32) ---
VA=  0 -> PA=  0
VA= 10 -> PA= 10
[EXCEPCION] VA=63 viola bounds=32
[EXCEPCION] VA=64 viola bounds=32
[EXCEPCION] VA=100 viola bounds=32

```

El proceso A no puede acceder a direcciones del proceso C porque sus espacios de direcciones son distintos, estos están definidos por dos límites el inferior que es el base le cual es inclusivo y el superior que es el bounds que no es inclusivo, tenido esto en cuenta para el proceso C tenemos base de 0 y bounds de 32 y para el proceso A tenemos base de 32 y bounds de 64 por lo que en sus espacios de direcciones no comparten ninguna dirección.

3. ¿Cual es la limitación principal del esquema base & bounds que motiva el surgimiento de la segmentación?

Su mayor limitación es algo conocido como fragmentación interna, esta se refiere a los huecos que quedan entre los elementos del mapa de memoria y que no pueden ser aprovechados para crear más procesos.

Segmentación

4.1 Traducción manual con tabla de segmentos

Dado un espacio de 14 bits (2 bits de selector + 12 bits de offset), use la siguiente tabla de segmentos:

Tabla de segmentos:

Segmento	Base (PA)	Tamaño	Crecimiento	Selector
Code	0x4000	2 KB	positivo	00
Heap	0x6000	3 KB	positivo	01
Stack	0x2800	2 KB	negativo	11

2 KB = 0x800 bytes

3 KB = 0xC00 bytes

Traduzca manualmente y complete la tabla:

VA (hex)	Selector	Offset	Segmento	PA o Excepción
0x03A0	00	0x3A0	Code	0x43A0
0x1800	01	0x800	Heap	0x6800

0x3C00	11	0xC00	Stack	Excepción de segmentación
0x0C00	00	0xC00	Code	Excepción de segmentación
0x2200	10	—	???	Excepción por segmento inválido

1. Muestre el cálculo paso a paso para cada VA.

A- Traducción:

VA: 0x03A0

Selector: 00

offset: 0x03A0

Validación:

0x03A0 < 0x0800

Cálculo:

PA = 0x4000 + 0x03A0

PA = 0x43A0

B- Traducción:

VA: 0x1800

Selector: 01

offset: 0x0800

Validación:

0x0800 < 0x0C00

Cálculo:

PA = 0x6000 + 0x0800

PA = 0x6800

C- Traducción:

VA: 0x3C00

Selector: 11

offset: 0x0C00

Validación:

El stack tiene tamaño 0x0800

Como:

$0x0C00 > 0x0800$

En este caso el offset que sería 0x0C00 excede el tamaño permitido.

Resultado:

Excepción de segmentación

D- Traducción:

VA: 0x0C00

Selector: 00

offset: 0x0C00

Validación:

El segmento Code tiene tamaño 0x0800

Como:

$0x0C00 > 0x0800$

En este caso el offset que sería 0x0C00 excede el tamaño permitido.

Resultado:

Excepción de segmentación

E- Traducción:

VA: 0x2200

Selector: 10

Validación:

No existe ningún segmento asociado al selector 10 en la tabla.

Resultado:

Excepción por segmento inválido

2. ¿Por qué el Stack crece en dirección negativa? ¿Qué ajuste especial requiere la fórmula al calcular el PA?

El stack crece en dirección negativa porque, en la mayoría de arquitecturas, las nuevas llamadas a funciones y variables locales se almacenan hacia direcciones de memoria menores. Debido a esto, el cálculo de la dirección física no se realiza como $\text{Base} + \text{Offset}$, sino restando desde el límite superior del segmento.

La fórmula utilizada es:

$PA = Base - (Tamaño - Offset)$

Esto permite que el stack pueda ir hacia abajo en memoria.

3. ¿Qué ventaja tiene la segmentación frente a base & bounds en cuanto a utilización de la memoria física?

La segmentación permite dividir el espacio de direcciones en múltiples regiones independientes como code, heap y stack, cada una con tamaños y ubicaciones diferentes en memoria física. Esto mejora el aprovechamiento de memoria física porque no es necesario reservar un único bloque contiguo grande para todo el proceso. Cada segmento puede ubicarse en diferentes posiciones de memoria según la disponibilidad.

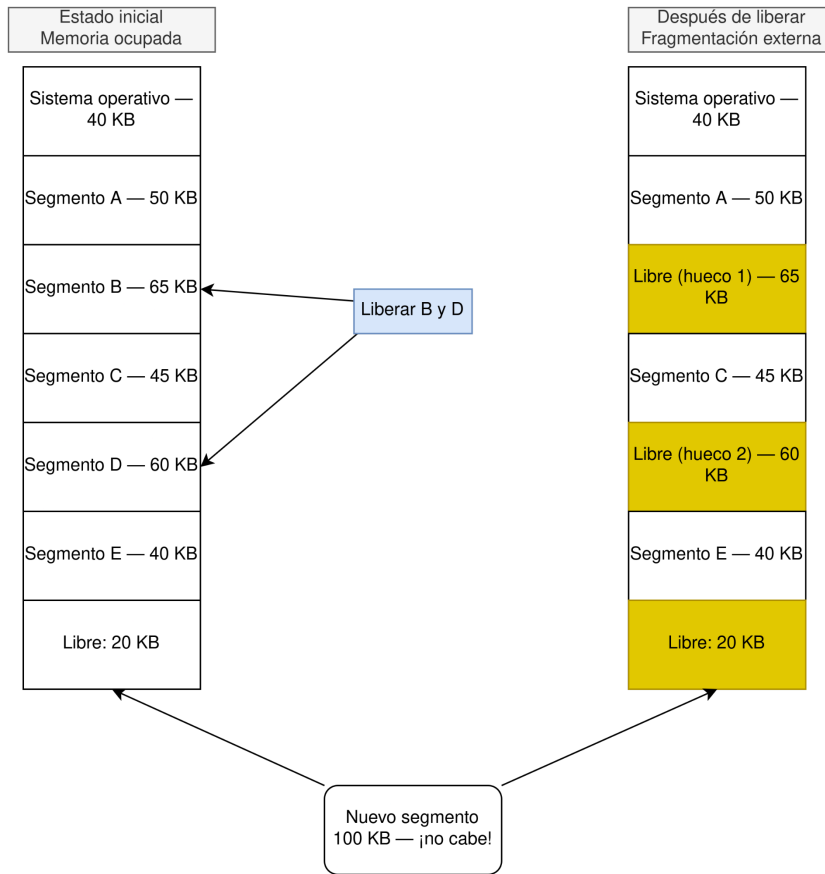
En cambio, el esquema simple de base & bounds requiere una única región contigua para todo el espacio del proceso.

4. ¿Qué es la fragmentación externa? ¿Por qué surge con segmentación? Ilustre con un diagrama de bloques de memoria.

La fragmentación externa ocurre cuando la memoria libre queda dividida en pequeños espacios dispersos entre segmentos ocupados. Aunque la suma de memoria libre pueda ser suficiente, puede no existir un bloque contiguo lo bastante grande para alojar un nuevo segmento.

Esto sucede en segmentación porque los segmentos tienen tamaños variables y se cargan o liberan dinámicamente.

Ejemplo:



Tras liberar B y D, quedan tres huecos: 65 KB, 60 KB y 20 KB al final, sumando 145 KB libres en total. Aun así, un nuevo proceso que necesite 100 KB contiguos no puede ser atendido, ya que ningún hueco individual alcanza ese tamaño la memoria está fragmentada en espacios discontinuos separados por los segmentos ocupados A, C y E.

Paginación

5.1 Actividad: Cálculo de la tabla de páginas

Considere el sistema:

Parámetro	Valor
Espacio virtual	32 bits

Tamaño de página	4 KB = 2^{12} bytes
Espacio físico	20 bits
Tamaño de PTE	4 bytes

1. **¿Cuántos bits se necesitan para el VPN y cuantos para el offset? Muestre el cálculo.**

El tamaño de la página es: 4 KB = 4096 bytes = 2^{12} bytes

$$\text{num pages} = \frac{2^{32}}{2^{12}} = 2^{20}$$

$$2^{n(\text{VPN})} = \text{num pages} = 2^{20}$$

$$n(\text{VPN}) = 20$$

$$2^{n(\text{offset})} = \text{size}(\text{page})$$

$$2^{n(\text{offset})} = 2^{12}$$

$$n(\text{offset}) = 12$$

la cantidad de bits que se necesitan para el VPN debe ser la suficiente para poder representar los num pages, en este caso se necesitan 20 bits

2. **¿Cuántas entradas tiene la tabla de páginas de un proceso?**

Cada combinación posible del VPN necesita una entrada en la tabla de páginas.

Como el VPN tiene 20 bits:

$$\text{Entradas} = 2^{20}$$

$$\text{Entradas} = 1,048,576$$

La tabla de páginas tiene 1,048,576 entradas.

3. **¿Cuánta memoria ocupa la tabla de páginas completa? ¿Es razonable ese tamaño para cada proceso?**

$$\text{size}(\text{Page table}) = \text{size}(\text{PTE}) * \text{num pages}$$

$$\text{size}(\text{Page table}) = 4 * 2^{20} = 2^{22}$$

$$\text{size}(\text{Page table}) = 4,194,304 \text{ bytes} = 4 \text{ MB}$$

Resultado:

La tabla de páginas completa ocupa aproximadamente 4 MB por proceso.

Respecto a si es razonable al analizar los resultados vemos que tenemos un espacio físico de 1MB mientras que necesitamos destinar 4 MB para almacenar la page table, osea que se necesitan 4 veces el espacio físico para guardar la page table este tamaño es bastante grande por lo que no sería razonable.

4. ¿Cuántos bits necesita el PFN dentro de la PTE? ¿Qué información almacena los bits restantes? Mencione al menos 3 bits de control y su función.

El espacio físico: 2^{20} bytes = 1 MB

Cada página ocupa 4 KB

offset es de 12 bits

$$\text{num frames} = \frac{\text{size}(PM)}{\text{size}(Frame)}$$

$$\text{num frames} = \frac{2^{20}}{2^{12}} = 2^8$$

$$2^{n(PFN)} = \text{num frames}$$

$$2^{n(PFN)} = 2^8$$

$$n(PFN) = 8$$

Los bits restantes de la PTE se utilizan como bits de control.

Bits de control:

- Los bits más significativos que acompañan al PFN en la PTE, se dividen en lo siguiente:
- Bit de validez(V): indica si la traducción es válida o sea si un PTE puede o no ser usado
- Bit de referencia(R): Indica si una página ha sido accedida
- Bit de presencia(P): Indica si una página está en memoria física o si está en el disco duro
- Bit sucio(M): Indica si la página ha sido modificada
- Bits de protección(Prot): definen las características del frame respecto a los permisos que se tienen sobre este pedazo de memoria (Read-Write-Execute)

5.2 Actividad: Simulador de paginación

En este punto solo debíamos guardar el archivo paging_sim.c

5.3 Actividad: Simulador — Análisis

1. Compile y ejecute el simulador. Muestre la salida completa.

None

./paging_sim

VA

VPN

Offset

PFN

PA

```
-----  
VA=0x00 VPN= 0 Offset= 0 -> PFN= 3 PA=0x30  
VA=0x0F VPN= 0 Offset=15 -> PFN= 3 PA=0x3F  
VA=0x20 VPN= 2 Offset= 0 -> PFN= 7 PA=0x70  
VA=0x35 VPN= 3 Offset= 5 -> PFN= 2 PA=0x25  
VA=0x10 VPN= 1 Offset= 0 -> PAGE FAULT (pagina no presente)  
VA=0xA3 VPN=10 Offset= 3 -> PFN= 4 PA=0x43  
VA=0xC8 VPN=12 Offset= 8 -> PFN= 6 PA=0x68  
VA=0xF0 VPN=15 Offset= 0 -> PAGE FAULT (pagina no presente)
```

2. ¿Qué ocurre con las VAs 0x10 y 0xA3? ¿Qué debería hacer el SO real ante un page fault?

Con la VA 0xA3, no produce un error y se puede traducir correctamente, respecto a la VA 0x10 se produce un page fault, esto significa que el bit de presencia para esta página está en 0, lo que se debe hacer en un caso real es verificar si la referencia es válida, luego se localiza la página en el disco duro, se confirma que sea válida, en caso de que sea válida se debe traer la página del disco para la memoria física a un frame, en caso de que no se tenga espacio disponible se usa una política de reemplazo para reemplazar uno de los frames.

3. ¿Cuántos accesos a memoria física requiere completar una instrucción load con tabla de páginas de un solo nivel? ¿Por qué es costoso y qué solución de hardware existe?

Con una tabla de páginas de un solo nivel se requieren dos accesos a memoria física:

1. Un acceso para leer la Page Table Entry.
2. Otro acceso para leer el dato real en memoria física.

Esto incrementa el tiempo de acceso porque cada referencia de memoria necesita realizar una operación de acceso que es costosa. La solución de hardware utilizada es el Translation Lookaside Buffer (TLB), una memoria caché utilizada para almacenar traducciones recientes.

Cuando ocurre un TLB hit, la traducción se obtiene directamente desde la TLB y solo se requiere un acceso a memoria física para obtener el dato.

4. ¿Que ventaja tiene la paginación sobre la segmentación en cuanto al fenómeno de fragmentación?

La principal ventaja de la paginación es que elimina la fragmentación externa. Como todas las páginas y frames tienen tamaño fijo, cualquier frame libre puede utilizarse para almacenar cualquier página, evitando huecos dispersos de tamaños variables en memoria física. En cambio, la segmentación utiliza segmentos de tamaño variable, lo que produce fragmentación externa con el tiempo.

Gestión de espacio libre

6.1 Actividad: Simulación de estrategias de asignación

Estado inicial de la lista libre:

Dirección inicio	Tamaño (bytes)
0x0100	100
0x0200	500
0x0400	200
0x0500	300
0x0700	600

Solicitudes en orden: malloc(212) · malloc(417) · malloc(98) · malloc(426)

1. Para cada solicitud indique que bloque asigna first fit. Muestre la lista libre resultante tras las 4 asignaciones.

First Fit selecciona el primer bloque libre que sea suficientemente grande para satisfacer cada solicitud.

Solicitud 1: malloc(212)

El primer bloque capaz de alojar 212 bytes es:

0x0200 = 500 bytes

Se asignan 212 bytes y quedan:

500 - 212 = 288 bytes libres

Nueva lista:

Dirección	Tamaño
0x0100	100
0x02D4	288
0x0400	200
0x0500	300
0x0700	600

Solicitud 2: malloc(417)

Los bloques de 100, 288, 200 y 300 bytes no alcanzan.

Se utiliza:

0x0700 = 600 bytes

Quedan:

600 - 417 = 183 bytes

Nueva lista:

Dirección	Tamaño
0x0100	100
0x02D4	288
0x0400	200
0x0500	300
0x08A1	183

Solicitud 3: malloc(98)

El primer bloque suficiente es:

0x0100 = 100 bytes

Quedan:

100 - 98 = 2 bytes

Nueva lista:

Dirección	Tamaño
0x0162	2
0x02D4	288
0x0400	200
0x0500	300
0x08A1	183

Solicitud 4: malloc(426)

Ningún bloque disponible tiene tamaño suficiente.

Resultado:

La asignación falla.

Lista final con First Fit:

Dirección	Tamaño
0x0162	2
0x02D4	288
0x0400	200
0x0500	300
0x08A1	183

2. Repita con best fit. ¿Cambia el resultado?

El best fit primero mira en qué bloques se puede alojar lo que se está solicitando y escoge de estos el que tenga menor tamaño

Solicitud 1: malloc(212)

Los bloques que tienen suficiente espacio para reservar son:

- 0x0200 con un tamaño de 500 bytes
- 0x0500 con un tamaño de 300 bytes
- 0x0700 con un tamaño de 600 bytes

Elegimos el bloque 0x0500 porque es el que tiene el menor tamaño, asignamos los 212 bytes y nos quedarían 88 bytes libres para ese bloque, la nueva lista quedaría de la siguiente manera

Dirección inicio	Tamaño (bytes)
0x0100	100
0x0200	500
0x0400	200
0x05d4	88
0x0700	600

Solicitud 2: malloc(417):

Los bloques que tienen suficiente espacio para reservar son:

- 0x0200 con un tamaño de 500 bytes
- 0x0700 con un tamaño de 600 bytes

Elegimos el bloque 0x0200 porque es el que tiene el menor tamaño, asignamos los 417 bytes y nos quedan 83 bytes libres para este bloque y la nueva lista sería la siguiente

Dirección inicio	Tamaño (bytes)
------------------	----------------

0x0100	100
0x0200	83
0x0400	200
0x05d4	78
0x0700	600

Solicitud 3: malloc(98)

Los bloques que tienen suficiente espacio para reservar son:

- 0x0100 con un tamaño de 100 bytes
- 0x0400 con un tamaño de 200 bytes
- 0x0700 con un tamaño de 600 bytes

Se elige el bloque 0x0100 porque es el que tiene el menor tamaño, asignamos los 98 bytes y nos quedarían 2 bytes libres en el bloque y la nueva lista sería la siguiente

Dirección inicio	Tamaño (bytes)
0x0162	2

0x0200	83
0x0400	200
0x05d4	78
0x0700	600

Solicitud 4: malloc(426)

Los bloques que tienen suficiente espacio para reservar son:

- 0x0700 con un tamaño de 600 bytes

Se elige el bloque 0x0700 porque es el único con el tamaño necesario donde asignamos los 426 bytes y nos quedarían 174 bytes libres en el bloque y la nueva lista sería la siguiente

Dirección inicio	Tamaño (bytes)
0x0162	2
0x0200	83
0x0400	200

0x05d4	78
0x08aa	174

3. ¿Cuál estrategia genera más fragmentación externa en este caso? ¿Cuál la minimiza?

En este caso, First Fit genera más fragmentación externa porque deja bloques medianos dispersos y desperdicia parte del bloque de 500 bytes desde la primera asignación.

Best Fit minimiza mejor la fragmentación externa porque intenta usar el bloque más ajustado posible para cada solicitud, preservando bloques grandes para futuras asignaciones. Mientras que Best Fit también puede producir muchos bloques pequeños difíciles de reutilizar.

4. ¿Qué es el coalescing? Ilustre un caso donde su ausencia provoca que una solicitud de 250 bytes falle aunque haya suficiente memoria total libre.

El coalescing consiste en que al momento de liberar un bloque de memoria, se debe de revisar si los bloques cercanos respecto a las direcciones también son memoria libre y en el caso de que lo sean se deben unir los bloques para que de esta forma el bloque de memoria libre sea lo más grande posible

Ejemplo

Inicialmente tenemos los siguiente bloques de memoria

Bloque	Tamaño
Libre	100 bytes
Ocupado	200 bytes
Libre	150 bytes
Ocupado	120 bytes

Luego realizamos la liberación del cuarto bloque, en un sistema sin coalescing obtendremos lo siguiente

Bloque	Tamaño
Libre	100 bytes
Ocupado	200 bytes
Libre	150 bytes
Libre	120 bytes

La memoria libre total:
 $100 + 150 + 120 = 370$ bytes

Sin embargo, si los bloques de 150 y 120 bytes no se fusionan, una solicitud de:

`malloc(250)`

Fallará porque no existe un bloque contiguo suficientemente grande. En cambio con coalescing los bloques libres contiguos podrían combinarse para satisfacer la solicitud.

5. ¿Qué es la fragmentación interna? ¿Cuándo aparece típicamente al usar un slab allocator?

La fragmentación interna ocurre cuando un proceso utiliza solamente una parte del bloque asignado y el espacio restante queda desperdiciado dentro del mismo bloque.

En un slab allocator esto aparece típicamente porque los objetos se almacenan en bloques de tamaño fijo. Por ejemplo, si el allocator maneja bloques de 64 bytes y un objeto necesita únicamente 50 bytes, los 14 bytes restantes quedan inutilizados, produciendo fragmentación interna.

6.2 Actividad: Fragmentación

En este punto solo debemos guardar el archivo `fragmentation.c`

Se compila con:

None

```
gcc -Wall -o fragmentation fragmentation.c
```

Salida al compilar

Ejecución

```
None
./fragmentation
malloc( 16) -> 0x55c65cf602a0
malloc( 32) -> 0x55c65cf606d0
malloc( 64) -> 0x55c65cf60700
malloc(128) -> 0x55c65cf60750
malloc(256) -> 0x55c65cf607e0
malloc(512) -> 0x55c65cf608f0
malloc(1024) -> 0x55c65cf60b00
malloc(512) -> 0x55c65cf60f10
malloc(256) -> 0x55c65cf61120
malloc(128) -> 0x55c65cf61230

Liberando bloques en indices pares...

malloc(1500) -> 0x55c65cf612c0 [exito]
```

6.3 Actividad: Fragmentación en glibc — Análisis

1. ¿Son consecutivas en memoria las direcciones asignadas? ¿Qué patrón de separación observa entre bloques contiguos?

La diferencia entre la dirección 2 y la dirección 1 es de 1072

La diferencia entre la dirección 3 y la dirección 2 es de 48

La diferencia entre la dirección 4 y la dirección 3 es de 80

La diferencia entre la dirección 5 y la dirección 4 es de 144

La diferencia entre la dirección 6 y la dirección 5 es de 272

La diferencia entre la dirección 7 y la dirección 6 es de 528

La diferencia entre la dirección 8 y la dirección 7 es de 1040

La diferencia entre la dirección 9 y la dirección 8 es de 528

La diferencia entre la dirección 10 y la dirección 9 es de 272

Las direcciones asignadas usando malloc parecen cercanas pero en realidad no son consecutivas entre sí como se puede observar en el cálculo anterior de la diferencia entre las direcciones. Respecto al patrón de separación podemos ver que la diferencia entre las direcciones es para todos los casos excepto el primero igual al tamaño reservado sumado junto a 16 bytes, los cuales suelen representar la metadata del bloque o una alineación a 16 bytes.

2. ¿Tiene éxito la asignación final de 1500 bytes? Explique el resultado en términos de fragmentación.

Sí, la asignación final fue exitosa:

None

```
malloc(1500) -> 0x55c65cf612c0 [exito]
```

Aunque se liberaron varios bloques intermedios, glibc logró encontrar suficiente espacio libre para satisfacer la solicitud de 1500 bytes.

Esto indica que:

1. el heap todavía disponía de memoria libre contigua suficiente
2. el allocator pudo extender el heap o reutilizar regiones disponibles
3. la fragmentación externa no era todavía lo suficientemente severa para impedir la asignación

En este caso sí existe fragmentación, porque quedaron huecos tras liberar los índices pares, pero no fue suficiente para provocar un fallo de asignación.

3. Consulta: ¿Cuál es la diferencia entre el allocator de usuario (malloc/glibc) y el del kernel (buddy system, slab)? ¿Por qué existen dos niveles de gestión de memoria?

El allocator de usuario, como malloc() en glibc, administra el heap de cada proceso y trabaja sobre memoria virtual entregada por el sistema operativo.

Su función es:

1. dividir bloques
2. reutilizar memoria liberada
3. manejar fragmentación
4. entregar memoria dinámica a programas de usuario

En cambio, el allocator del kernel administra directamente la memoria física del sistema operativo.

El kernel utiliza mecanismos como:

- Buddy System: divide memoria en bloques de tamaños potencia de dos para asignaciones grandes y eficientes.
- Slab Allocator: mantiene cachés de objetos pequeños reutilizables para reducir overhead y fragmentación.

Existen dos niveles de gestión porque las necesidades son diferentes:

- Los programas de usuario necesitan una interfaz flexible y sencilla como malloc().
- El kernel necesita control directo, eficiencia y administración global de toda la RAM física.

De esta manera, el sistema operativo administra memoria física a bajo nivel, mientras que cada proceso administra su propio heap a nivel de usuario.

TLBs — Translation Lookaside Buffer

Compilación con la opción de optimización

```
None
gcc -O -o tlb_locality tlb_locality.c
```

7.1 Actividad: Localidad y TLB — Análisis

1. ¿Cuántas veces más lento es el acceso aleatorio frente al secuencial? Muestre el promedio de 3 ejecuciones de tlb locality.

Primera salida

```
None
./tlb_locality
Secuencial :      3.17 ms (sum=8796090925056)
Aleatorio :      44.89 ms (sum=8796090925056)
```

Segunda salida

```
None
./tlb_locality
Secuencial :      4.98 ms (sum=8796090925056)
Aleatorio :      46.94 ms (sum=8796090925056)
```

Tercera salida

```
None
./tlb_locality
Secuencial :      4.98 ms (sum=8796090925056)
Aleatorio :      46.94 ms (sum=8796090925056)
```

Promedio para secuencial: 4.38

Promedio para aleatorio: 46.26

Al utilizar el acceso secuencial somos 10.56 más rápidos que usando el acceso aleatorio

2. Explique con el modelo del TLB por qué el acceso aleatorio es más lento. ¿Qué ocurre con el hit rate en cada caso?

El acceso secuencial presenta alta localidad espacial porque los datos consecutivos suelen encontrarse dentro de las mismas páginas o en páginas cercanas. Esto permite que las traducciones de VPN a PFN permanezcan en el TLB, ocurran muchos TLB hits y se reduzcan los accesos a la tabla de páginas. Por esta razón, el acceso secuencial es mucho más rápido.

Por otro lado, el acceso aleatorio rompe la localidad espacial y temporal, ya que el programa accede constantemente a páginas diferentes. Esto provoca más TLB misses, aumentando las consultas a la tabla de páginas causando un mayor tiempo de traducción de direcciones. Por lo tanto, el acceso secuencial produce un alto hit rate, mientras que el acceso aleatorio genera un bajo hit rate.

3. Si el tamaño de página fuera 64 KB en lugar de 4 KB, ¿mejoraría o empeoraría la situación con accesos aleatorios? Justifique desde el punto de vista del TLB y del uso de memoria.

Con páginas de 64 KB, cada entrada del TLB cubriría una región mucho más grande de memoria. Esto mejoraría el rendimiento de los accesos aleatorios porque serían necesarias menos páginas para cubrir el arreglo completo, el TLB podría almacenar traducciones útiles para regiones más amplias y disminuiría la cantidad de TLB misses.

Sin embargo, también existen desventajas. El uso de páginas más grandes incrementa la fragmentación interna, provoca un mayor desperdicio de memoria dentro de cada página. En general, para accesos aleatorios el TLB podría beneficiarse de páginas más grandes, aunque con un mayor costo en el uso de memoria.

7.2 Actividad: Comportamiento de los TLB

Responda conceptualmente estas preguntas:

1. Un TLB con 64 entradas (fully associative) y páginas de 4 KB. ¿Cuánta memoria puede cubrir sin generar misses? ¿Es suficiente para un proceso moderno típico?

Para una TLB con esas características podríamos guardar la traducción para 64 páginas, teniendo en cuenta que cada página tiene un tamaño de 4 KB, para obtener la cantidad de memoria que puede cubrir sin generar misses debemos de multiplicar el tamaño de página por la cantidad de entradas

$$\text{Cantidad de memoria que puede cubrir} = 64 * 4kb = 256KB$$

Respecto a si es suficiente se considera que para procesos modernos típicos no sería suficiente porque estos suelen tener espacios de memoria del orden de varios MB o incluso de GB, entonces con una TLB tan pequeña se generarían muchos TLB misses de forma que no aportaría prácticamente al rendimiento.

2. Consulte: ¿Qué es un TLB shutdown y en qué situación ocurre en sistemas multiprocesador? ¿Por qué es una operación costosa?

Un TLB shutdown ocurre cuando el sistema operativo necesita invalidar entradas del TLB en múltiples CPUs. Esto sucede normalmente cuando cambia una tabla de páginas, se libera memoria, se modifica un mapeo virtual o cambian los permisos de acceso de una página.

En sistemas multiprocesador, cada CPU puede tener almacenadas copias distintas de las traducciones de memoria en su propio TLB. Por eso, cuando el sistema operativo modifica una traducción, debe asegurarse de que todas las CPUs eliminen las entradas antiguas para evitar inconsistencias.

Para lograrlo, el sistema operativo envía interrupciones entre procesadores (IPIs) con el fin de obligar a cada núcleo a invalidar sus entradas del TLB. Esta operación es costosa porque requiere sincronización entre CPUs, interrumpe temporalmente la ejecución normal de los procesos, invalida traducciones útiles almacenadas en caché y además genera overhead de comunicación entre los distintos núcleos del procesador.

3. Explique la diferencia entre TLB gestionado por hardware (CISC/x86) y por software (RISC/MIPS). ¿Cuál ofrece mayor flexibilidad al diseñador del SO y por qué?

En arquitecturas como x86, el TLB es manejado principalmente por hardware. Cuando ocurre un TLB miss, el hardware consulta automáticamente la tabla de páginas, carga la traducción correspondiente en el TLB y luego reanuda la ejecución del programa. Este mecanismo simplifica el diseño del sistema operativo y hace que la gestión de memoria sea más transparente para el programador. En arquitecturas como las MIPS clásicas, el TLB puede ser manejado por software. En este caso, cuando ocurre un miss, se genera una excepción y el sistema operativo toma el control para decidir cómo resolverla. Luego, el propio sistema operativo inserta manualmente la traducción adecuada dentro del TLB.

El manejo por software ofrece mayor flexibilidad al diseñador del sistema operativo, ya que permite implementar distintos formatos de tablas de páginas, facilita el uso de políticas personalizadas de reemplazo y brinda más control sobre la administración de memoria. Sin embargo, también aumenta la complejidad del sistema operativo y puede introducir más overhead durante la ejecución.